

Chapter 1: Introduction

Mobile computing and Android Market

Mobile computing refers to the use of portable computing devices, such as smartphones, tablets, and laptops, in conjunction with mobile software applications and wireless networks to enable users to access information, perform tasks, and communicate from anywhere, at any time. It involves the interaction between mobile hardware, mobile software, and network connectivity, allowing users to be productive and stay connected while on the move. Mobile computing has revolutionized the way people work, communicate, and access information. It has paved the way for a plethora of innovative applications and services, transforming various industries and enhancing personal and professional productivity. Before the Android operating system was introduced, there were several mobile phone manufacturers and operating systems in the market. Some of the notable mobile phone companies and platforms before Android's launch include:

1. **Nokia:** Nokia was one of the dominant players in the mobile phone industry before the smartphone era. They used their proprietary operating system called Symbian for many of their devices.
2. **Motorola:** Motorola was a prominent mobile phone manufacturer known for its iconic flip phones and other mobile devices. They used various operating systems, including their proprietary OS and some phones using Windows Mobile.
3. **BlackBerry:** BlackBerry was popular for its smartphones that featured physical keyboards. They used their proprietary operating system known as BlackBerry OS.
4. **Palm:** Palm was known for its Palm OS, which was used in devices like the Palm Pilot and Treo smartphones. Palm OS was widely used before the smartphone boom.
5. **Sony Ericsson:** Sony Ericsson was a joint venture between Sony and Ericsson. They produced mobile phones using various operating systems, including their proprietary platforms and some phones using Windows Mobile.
6. **Microsoft:** Microsoft developed Windows Mobile, a mobile operating system that was used by various manufacturers, including HTC, Samsung, and others, before the rise of Android.

7. **LG:** LG was another mobile phone manufacturer that used different operating systems, including their proprietary platforms and some phones using Windows Mobile.
8. **Samsung:** Samsung produced a range of mobile phones with various operating systems, including their proprietary OS and some phones using Windows Mobile.

These companies and operating systems were prevalent before the introduction of Android in 2008. Android's entry into the market significantly changed the smartphone landscape and eventually became the dominant mobile operating system globally.

What is an android?

Android is a mobile operating system that is based on a modified version of Linux. It was originally developed by a startup of the same name, Android, Inc. In 2005, as part of its strategy to enter the mobile space, Google purchased Android and took over its development work (as well as its development team).

Google wanted Android to be open and free; hence, most of the Android code was released under the open-source Apache License, which means that anyone who wants to use Android can do so by downloading the full Android source code. Moreover, vendors (typically hardware manufacturers) can add their own proprietary extensions to Android and customize Android to differentiate their products from others. This simple development model makes Android very attractive and has thus piqued the interest of many vendors. This has been especially true for companies affected by the phenomenon of Apple's iPhone, a hugely successful product that revolutionized the smartphone industry. Such companies include Motorola and Sony Ericsson, which for many years have been developing their own mobile operating systems. When the iPhone was launched, many of these manufacturers had to scramble to find new ways of revitalizing their products. These manufacturers see Android as a solution — they will continue to design their own hardware and use Android as the operating system that powers it.

The main advantage of adopting Android is that it offers a unified approach to application development. Developers need only develop for Android, and their applications should be able to run on numerous different devices, as long as the devices are powered using Android. In the world of smartphones, applications are the most important part of the success chain. Device manufacturers therefore see Android as their best hope to challenge the onslaught of the iPhone, which already commands a large base of applications.

Challenges Faced by the Mobile Market before the Rise of Android

Before the advent of Android, the mobile market was dominated by different operating systems and platforms, several challenges plagued the market, hindering its growth and innovation. Some of the existing challenges faced by the mobile market before Android became widespread include:

- **Fragmentation:** There were numerous mobile operating systems, each with its own set of hardware requirements and software development kits. This fragmentation made it difficult for developers to create applications that worked seamlessly across diverse devices, limiting user experience and app availability.
- **Closed Ecosystems:** Many mobile platforms were closed ecosystems, controlled by specific companies. Users had limited choices, and developers faced barriers to entry, hindering the creation of unique and varied mobile experiences.
- **Limited Developer Support:** Developing applications for mobile devices was complex and often required extensive knowledge of specific programming languages and tools. The pool of developers capable of creating mobile applications was limited, slowing down the pace of app development and innovation.
- **High Development Costs:** Due to the fragmented nature of the market and the lack of standardized development tools, creating mobile applications was expensive. Developers had to invest in different development environments for different platforms so that Small developers and startups found it difficult to enter the market due to the financial burden, limiting the emergence of new, innovative apps.
- **Limited Innovation:** Closed ecosystems and limited developer support stifled innovation. There was a lack of diverse applications and features, as developers were restricted by the capabilities of existing platforms.
- **Lack of Interoperability:** Different mobile platforms had incompatible technologies and communication protocols. This lack of interoperability made it difficult for users to share data and use applications across different devices and platforms.
- **Security Concerns:** Security was a significant concern, as different platforms had varying levels of security measures. Users were often exposed to malware and other security threats.
- **High Entry Barriers:** Developing and distributing applications for mobile devices required significant resources, making it difficult for small developers and startups to enter the market.

The introduction of Android helped address many of these challenges by providing an open-source, customizable operating system that could run on a wide range of devices. Android's open nature, along with its robust development tools, encouraged innovation and made it easier for developers to create applications for a diverse audience.

Android Versions

Android has gone through quite a number of updates since its first release.



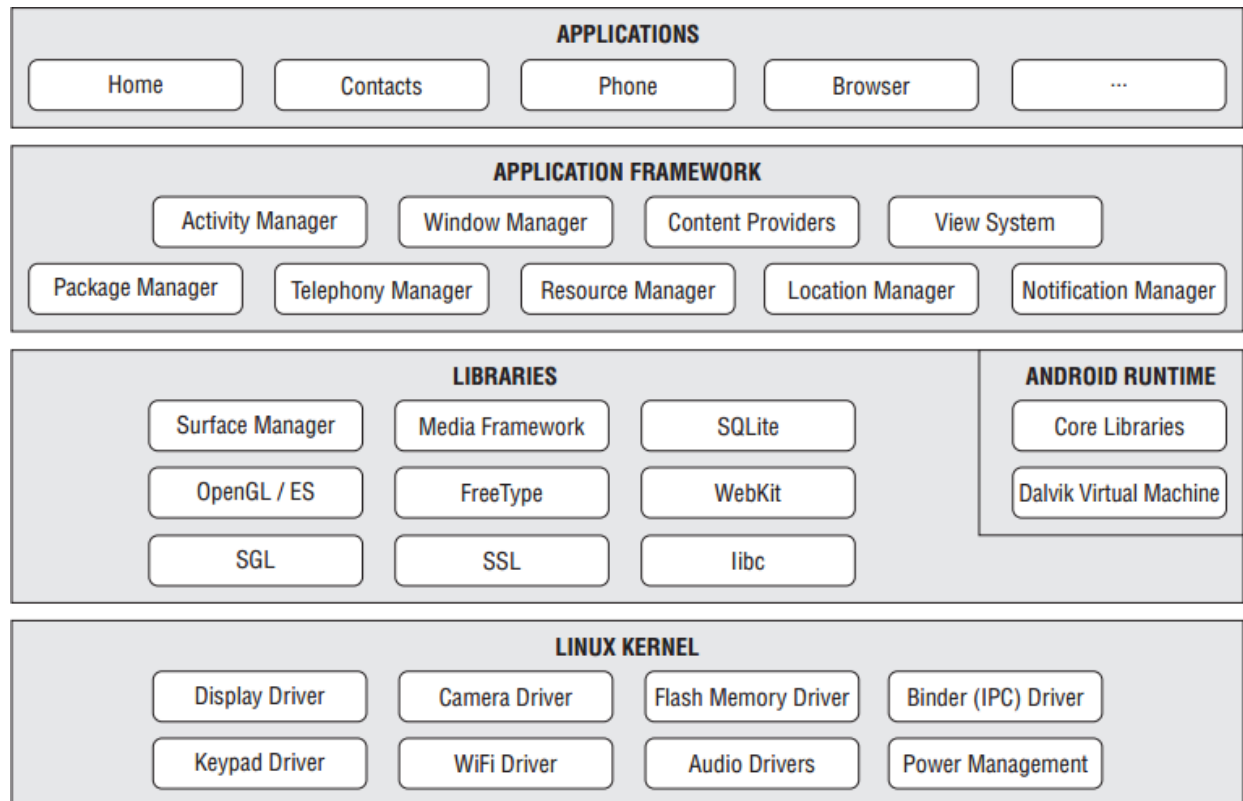
Features of Android

As Android is open source and freely available to manufacturers for customization, there are no fixed hardware and software configurations. However, Android itself supports the following features:

- Storage Uses SQLite, a lightweight relational database, for data storage.
- Connectivity — Supports GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth (includes A2DP and AVRCP), WiFi, LTE, and WiMAX.
- Messaging — Supports both SMS and MMS.
- Web browser— Based on the open-source WebKit, together with Chrome's V8 JS engine
- Media support— Includes support for the following media: H.263, H.264 (in 3GP or MP4 container), MPEG-4 SP, AMR, AMR-WB (in 3GP container), AAC, HE-AAC (in MP4 or 3GP container), MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, and BMP
- Hardware support— Accelerometer Sensor, Camera, Digital Compass, Proximity Sensor, and GPS
- Multi -touch— Supports multi-touch screens
- Multi -tasking— Supports multi-tasking applications
- Flash support— Android 2.3 supports Flash 10.1.
- Tethering — Supports sharing of Internet connections as a wired/wireless hotspot.

Architecture of Android

In order to understand how Android works, take a look at Figure 1-1, which shows the various layers that make up the Android operating system (OS).



The Android OS is roughly divided into five sections in four main layers:

- **Linux kernel** — This is the kernel on which Android is based. This layer contains all the lowlevel device drivers for the various hardware components of an Android device. At the bottom of the layers is Linux - Linux 3.6 with approximately 115 patches. This provides a level of abstraction between the device hardware and it contains all the essential hardware drivers like camera, keypad, display etc. Also, the kernel handles all the things that Linux is really good at such as networking and a vast array of device drivers, which take the pain out of interfacing to peripheral hardware.
- **Libraries** — These contain all the code that provides the main features of an Android OS. For example, the SQLite library provides database support so that an application can use it for data storage. The WebKit library provides functionalities for web browsing.

- `android.app` – Provides access to the application model and is the cornerstone of all Android applications.
 - `android.content` – Facilitates content access, publishing and messaging between applications and application components.
 - `android.database` – Used to access data published by content providers and includes SQLite database management classes.
 - `android.opengl` – A Java interface to the OpenGL ES 3D graphics rendering API.
 - `android.os` – Provides applications with access to standard operating system services including messages, system services and inter-process communication.
 - `android.text` – Used to render and manipulate text on a device display.
 - `android.view` – The fundamental building blocks of application user interfaces.
 - `android.widget` – A rich collection of pre-built user interface components such as buttons, labels, list views, layout managers, radio buttons etc.
 - `android.webkit` – A set of classes intended to allow web-browsing capabilities to be built into applications.
- **Android runtime**— At the same layer as the libraries, the Android runtime provides a set of core libraries that enable developers to write Android apps using the Java programming language. The Android runtime also includes the Dalvik virtual machine, which enables every Android application to run in its own process, with its own instance of the Dalvik virtual machine (Android applications are compiled into the Dalvik executables). Dalvik is a specialized virtual machine designed specifically for Android and optimized for battery-powered mobile devices with limited memory and CPU.
- **Application framework**— Exposes the various capabilities of the Android OS to application developers so that they can make use of them in their applications.
- **Applications**— At this top layer, you will find applications that ship with the Android device (such as Phone, Contacts, Browser, etc.), as well as applications that you download and install from the Android Market. Any applications that you write are located at this layer.

The Android market

As mentioned earlier, one of the main factors determining the success of a smartphone platform is the applications that support it. It is clear from the success of the iPhone that applications play a very vital role in determining whether a new platform swims or sinks. In addition, making these applications accessible to the general user is extremely important.

As such, in August 2008, Google announced the Android Market, an online application store for Android devices, and made it available to users in October 2008. Using the Market application that is preinstalled on their Android device, users can simply download third-party applications directly onto their devices. Both paid and free applications are supported on the Android Market, though paid applications are available only to users in certain countries due to legal issues.

Similarly, in some countries, users can buy paid applications from the Android Market, but developers cannot sell in that country. As an example, at the time of writing, users in India can buy apps from the Android Market, but developers in India cannot sell apps on the Android Market. The reverse may also be true; for example, users in South Korea cannot buy apps, but developers in South Korea can sell apps on the Android Market.

Anatomy of an Android Application

Now that you have created your first Hello World Android application, it is time to dissect the innards of the Android project and examine all the parts that make everything work. First, note the various files that make up an Android project in the Package Explorer.

The various folders and their files are as follows:

src — Contains the .java source files for your project. In this example, there is one file, MainActivity.java. The MainActivity.java file is the source file for your activity. You will write the code for your application in this file.

Android 4.4 library — This item contains one file, android.jar, which contains all the class libraries needed for an Android application.

gen — Contains the R.java file, a compiler-generated file that references all the resources found in your project. You should not modify this file.

assets — This folder contains all the assets used by your application, such as HTML, text files, databases,

res — This folder contains all the resources used in your application. It also contains a few other subfolders: drawable-<resolution>, layout, and values.

AndroidManifest.xml — This is the manifest file for your Android application. Here you specify the permissions needed by your application, as well as other features (such as intent-filters, receivers, etc.).

The main.xml file defines the user interface for your activity. Observe the following in bold:

```
<TextView
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:text="@string/hello"/>
```

The @string in this case refers to the strings.xml file located in the res/values folder. Hence, @string/hello refers to the hello string defined in the strings.xml file, which is “Hello World, MainActivity!”:

```
<?xmlversion="1.0"encoding="utf-8"?>
<resources>
    <string name="hello">Hello World, MainActivity!</string>
    <string name="app_name">HelloWorld</string>
</resources>
```

It is recommended that you store all the string constants in your application in this strings.xml file and reference these strings using the @string identifier. That way, if you ever need to localize your application to another language, all you need to do is replace the strings stored in the strings.xml file with the targeted language and recompile your application. Observe the content of the AndroidManifest.xml file:

```
<?xmlversion="1.0"encoding="utf-8"?>
<manifestxmlns:android="http://schemas.android.com/apk/res/android"
    package="net.learn2develop.HelloWorld"
    android:versionCode="1"
    android:versionName="1.0">
```



```
<applicationandroid:icon="@drawable/icon"
android:label="@string/app_name">

<activityandroid:name=".MainActivity"
android:label="@string/app_name">

<intent-filter>

<actionandroid:name="android.intent.action.MAIN"/>

<categoryandroid:name="android.intent.category.LAUNCHER"/>

</intent-filter>

</activity>

</application>

<uses-sdkandroid:minSdkVersion="9"/>

</manifest>
```

The AndroidManifest.xml file contains detailed information about the application:

- It defines the package name of the application as `net.learn2develop.HelloWorld`.
- The version code of the application is 1. This value is used to identify the version number of your application. It can be used to programmatically determine whether an application needs to be upgraded.
- The version name of the application is 1.0. This string value is mainly used for display to the user. You should use the format: `<major>.<minor>.<point>` for this value.
- The application uses the image named `icon.png` located in the `drawable` folder.
- The name of this application is the string named `app_name` defined in the `strings.xml` file.
- There is one activity in the application represented by the `MainActivity.java` file. The label displayed for this activity is the same as the application name.
- Within the definition for this activity, there is an element named `<intent-filter>`:
 - The action for the intent filter is named `android.intent.action.MAIN` to indicate that this activity serves as the entry point for the application.
 - The category for the intent filter is named `android.intent.category.LAUNCHER` to indicate that the application can be launched from the device's Launcher icon.
- Finally, the `android:minSdkVersion` attribute of the `<uses-sdk>` element specifies the minimum version of the OS on which the application will run.

As you add more files and folders to your project, Eclipse will automatically generate the content of R.java, which at the moment contains the following:

```
packagenet.learn2develop.HelloWorld;
publicfinalclassR{
publicstaticfinalclassattr{}
publicstaticfinalclassdrawable{
publicstaticfinalinticon=0x7f020000; }
publicstaticfinalclasslayout{
publicstaticfinalintmain=0x7f030000;}
publicstaticfinalclassstring{
publicstaticfinalintapp_name=0x7f040001;
publicstaticfinalinthello=0x7f040000;}
}
```

You are not supposed to modify the content of the R.java file; Eclipse automatically generates the content for you when you modify your project.

NOTE If you delete R.java manually, Eclipse will regenerate it for you immediately. Note that in order for Eclipse to generate the R.java file for you, the project must not contain any errors. If you realize that Eclipse has not regenerated R.java after you have deleted it, check your project again. The code may contain syntax errors, or your XML files (such as AndroidManifest.xml, main.xml, etc.) may not be well-formed.

Finally, the code that connects the activity to the UI (main.xml) is the **setContentView()** method, which is in the MainActivity.java file:

```
packagenet.learn2develop.HelloWorld;
importandroid.app.Activity;
importandroid.os.Bundle;
publicclassMainActivityextendsActivity{
/**Calledwhentheactivityisfirstcreated.*/
@Override
publicvoidonCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.main);
}}
```

Here, R.layout.main refers to the main.xml file located in the res/layout folder. As you add additional XML files to the res/layout folder, the file names will automatically be generated in the R.java file. The onCreate() method is one of many methods that are fired when an activity is loaded.